



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

CAPÍTULO 5: FUNCIONES, REFERENCIAS Y PUNTEROS



César Villacís, Margarita Zambrano, Sofía Bayona, Luis
Pastor, David Alvarado, Luis García, Kevin Topón

Universidad de las Fuerzas Armadas – ESPE
Quito, Ecuador

e-mail: {cjvillacis, mezambrano}@espe.edu.ec

Contenido

- **Punteros**

- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

PUNTEROS

Un puntero es una variable especial que almacena la dirección de memoria de una variable, para poder manipular el contenido de la variable a la que apunta (Granizo, E., 2016). Como cualquier tipo de variable, un puntero debe haber sido declarado antes de que pueda ser utilizado y ocupa 4 bytes de memoria. Un puntero se declara de acuerdo con la siguiente sintaxis:

Sintaxis:

Tipo_de_Dato *Nombre_del_Puntero;

Tipo_de_Dato

Tipo de dato definido por el lenguaje C/C++.

Operador de indirección, que indica que se creó un puntero.

Nombre_del_Puntero

Identificador que indica el nombre del puntero.

Nota:

Un puntero permite manipular la variable a la cual apunta, por lo tanto, puede ser utilizado para modificar el valor de la variable apuntada, para realizar operaciones de lectura y diferentes cálculos.

PUNTEROS

A continuación, se presentan algunos ejemplos que muestran la creación de punteros que apuntan a variables:

```
// Variables.  
int c = 7;  
char letra = 'M';  
bool veracidad = true;  
float ex = exp(2.0);  
  
// Punteros que apuntan a variables.  
int *ptr; ptr = &c;  
char *ptr1; ptr1 = &letra;  
bool *ptr2; ptr2 = &veracidad;  
float *ptr3; ptr3 = &ex;
```

También, se puede hacer que en una sola instrucción se realice la declaración e inicialización de un puntero, como se indica a continuación:

```
// Punteros que apuntan a variables.  
int *ptr = &c;  
char *ptr1 = &letra;  
bool *ptr2 = &veracidad;  
float *ptr3 = &ex;
```

Contenido

- Punteros
- **El Operador de Dirección (&)**
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

El Operador de Dirección (&)

El operador de dirección (&) es un operador unario que se utiliza tanto para almacenar la dirección de memoria de una variable, así como también para devolver la dirección de memoria de su operando (Davies, P., 1995). Por ejemplo, considerando las siguientes instrucciones:

```
a.  int c = 7;  
b.  int *ptr;  
c.  ptr = &c;
```

- a. Declaración e inicialización de la variable 'c' con el número '7'.
- b. El identificador 'ptr' es el nombre del puntero. El operador '*' significa que 'ptr' es un puntero. El puntero 'ptr' puede ser utilizado para apuntar variables de tipo 'int' (entero).
- c. El puntero 'ptr' es un puntero a entero inicializado con la dirección de la variable 'c'. Es muy común declarar y luego inicializar directamente al puntero, con lo que se obtienen las siguientes dos instrucciones en lugar de tres:

El Operador de Dirección (&)

- a. `int c = 7;`
- b. `int *ptr = &c;`

- a. Declaración e inicialización de la variable 'c' con el número '7'.
- b. Se crea un puntero a entero llamado 'ptr', utilizando el operador de indirección (*) y luego se lo inicializa con la dirección de la variable 'c'.

Contenido

- Punteros
- El Operador de Dirección (&)
- **El Operador de Indirección (*)**
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

El Operador de Indirección (*)

De acuerdo con (Davies, P., 1995), el operador de indirección (*) es un operador unario que, a más de ser utilizado como el operador de multiplicación, cumple con otras dos funciones muy importantes:

- Se utiliza para declarar o crear un puntero. Por ejemplo:

```
int *ptr;
```

- Se utiliza para devolver una expresión equivalente al valor contenido en la dirección apuntada por un puntero. Esta operación recibe el nombre de “desreferencia” ya que permite que un puntero pueda acceder y modificar el valor contenido actual de la variable apuntada por dicho puntero, por lo que se dice que el puntero está desreferenciado. Por ejemplo:

```
*ptr = 70;
```

- En esta instrucción se asigna al contenido de lo que apunta el puntero 'ptr' el valor de 70, utilizando el operador de indirección (*). Debido a que el puntero 'ptr' apunta a una variable, puede modificar el contenido de esa variable, utilizando el operador de indirección (*).

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- **El 'rvalue' y el 'lvalue' de una Variable y de un Puntero**
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

El 'rvalue' y el 'lvalue' de una Variable y de un Puntero

De acuerdo con (Davies, P., 1995), cuando en el lenguaje C/C++ se trabajan con variables y punteros se puede hablar del 'rvalue' y del 'lvalue' de una variable.

- a. El 'lvalue' de una variable es la dirección de memoria (localidad) donde la variable está almacenada.
- b. El 'rvalue' de una variable es el dato (contenido real) que la variable tiene. En otras palabras, es el valor de la variable.

Como se habló en la sección 1.4. la dirección de memoria donde se ubica una variable se representa en formato hexadecimal (hex). Por cuestiones didácticas vamos a utilizar valores en formato decimal (dec) para representar las direcciones de memoria de las variables.

Considerando las siguientes dos instrucciones explicadas anteriormente, vamos a analizar el 'rvalue' y el 'lvalue' de cada una de las variables:

```
int c = 7;  
int *ptr = &c;
```

El 'rvalue' y el 'lvalue' de una Variable y de un Puntero

- La variable 'c' tiene un rvalue igual a 7 y reside en la dirección de memoria o lvalue igual a hex 1000.
- El puntero 'ptr' reside en la dirección de memoria o lvalue igual a hex 3000 y en su rvalue tiene asignado la dirección de memoria de la variable 'c' que es igual a hex 1000. A continuación, se presenta en la Figura 5.5. el diagrama de bloques de los valores reales (rvalues) y de las localidades de la memoria (lvalues) de la variable 'c' y del puntero 'ptr'.

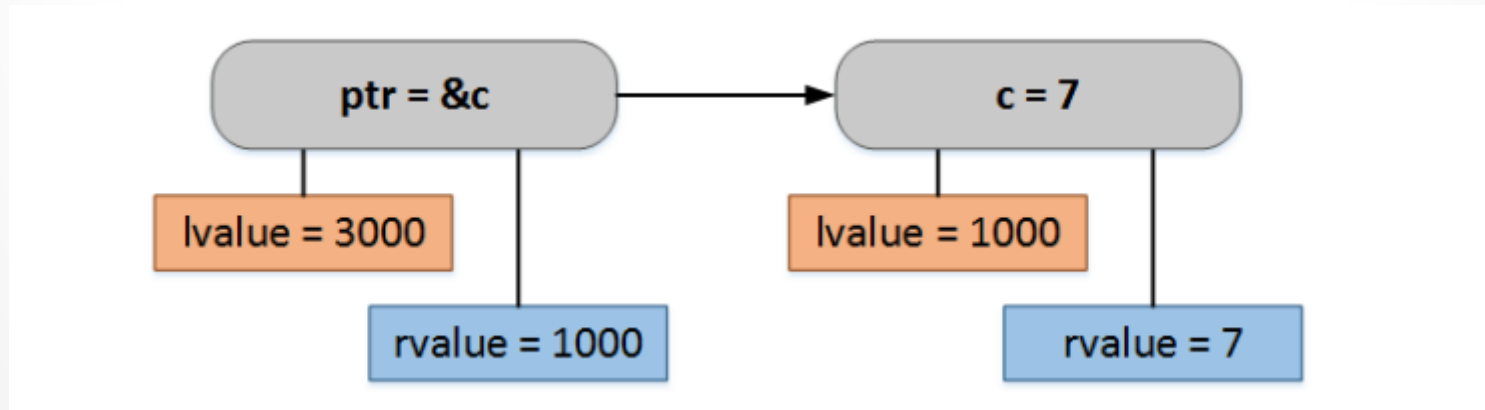


Figura 5.5 Diagrama de Bloques de los 'rvalue' y 'lvalue' de las variables. Fuente: Zambrano M., et al., 2025.

El 'rvalue' y el 'lvalue' de una Variable y de un Puntero

En la Figura 5.6. se presenta el mapa de la memoria RAM de la variable 'c' y del puntero 'ptr'

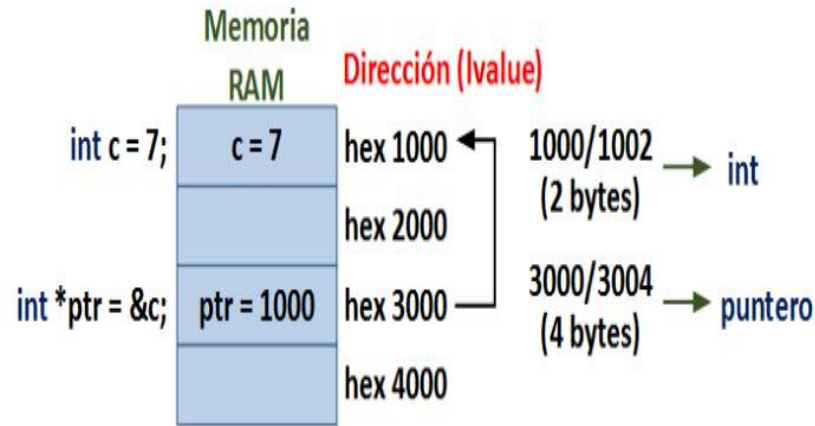


Figura 5.6 Mapa de la memoria RAM. Fuente: Zambrano M., et al., 2025.

Como se puede ver en el mapa de la memoria RAM la variable 'c' de tipo entero ocupa 2 bytes de memoria desde la dirección 1000 a la dirección 1002. El puntero 'ptr' en cambio ocupa 4 bytes de memoria desde la dirección 3000 a la dirección 3004 y apunta a la variable 'c'. El número de bytes que necesita una variable de tipo puntero (apuntador) no depende del tipo de dato al que apunta, por lo que un puntero siempre necesitará 4 bytes de memoria en plataformas de 32 bits, en cambio en plataformas de 6

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- **Problema 5.12**
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

Problema 5.12

Problema 5.12: Escribir un programa donde se crea una variable con el valor de 7 y a continuación se crea un puntero que apunta a esa variable. Luego, imprimir el valor de la variable y el contenido de lo que apunta el puntero. Después, modificar el contenido de lo que apunta el puntero con el número 70. Finalmente, imprimir nuevamente el contenido de la variable y el contenido de lo que apunta el puntero:

```
/******  
WinConsolaPrograma_5_12  
*****/  
  
// Librerías.  
#include <iostream>  
#include <cstdlib>  
  
using namespace std;  
  
// Función principal.  
int main()  
{  
    // Declaración e inicialización de la variable 'c' con el número 7.  
    int c = 7;
```

```

// Creación del puntero 'ptr' que va a apuntar a una variable de tipo
// entero, utilizando el operador de indirección (*).
int *ptr;
// Asignar al puntero 'ptr' la dirección de la variable 'c', utilizando
// el operador unario de dirección (&).
ptr = &c;

// Imprimir el contenido de la variable 'c', que en este caso es el
// número 7.
cout << "Variable 'c': " << c << endl;
// Imprimir el contenido de lo que apunta el puntero 'ptr', utilizando
// el operador de indirección (*). Debido a que el puntero 'ptr' apunta
// a la variable 'c', puede acceder al contenido de la variable 'c',
// utilizando el operador de indirección (*) y se imprime el valor
// que tiene almacenada la variable 'c' que es el número 7.
cout << "Contenido de lo que apunta el puntero 'ptr': " << *ptr << endl;

// Imprimir un salto de línea (INTRO).
cout << endl;

// Asignar al contenido de lo que apunta el puntero 'ptr' el valor de
// 70, utilizando el operador de indirección (*). Debido a que el
// puntero 'ptr' apunta a la variable 'c', puede modificar el contenido
// de la variable 'c', utilizando el operador de indirección (*).
*ptr = 70;

// Imprimir el valor de la variable 'c'. En este punto se comprueba
// que el puntero 'ptr' modifica el contenido de la variable 'c' que
// es la variable a la cual apunta.
cout << "Variable 'c': " << c << endl;
// Imprimir el contenido de lo que apunta el puntero 'ptr' nuevamente.
cout << "Contenido de lo que apunta el puntero 'ptr': " << *ptr << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}

```


Problema 5.12

```
C:\Users\cxvil\OneDrive\Documentos\Visual Studio 2019\Code Snippets\Visual C++\WinConsola...
Variable 'c': 7
Contenido de lo que apunta el puntero 'ptr': 7

Variable 'c': 70
Contenido de lo que apunta el puntero 'ptr': 70
Press any key to continue . . .
```

Salida 5.12. Salida del Programa. Fuente: Zambrano M., et al., 2025.

Como se puede ver en el programa 5.11, al apuntar el puntero 'ptr' a la variable 'c', este puntero puede acceder a la variable 'c' y modificar el contenido de la misma. El momento en que el puntero 'ptr' modifica el contenido de la variable 'c', la variable 'c' tiene un nuevo valor. En la Figura 5.12.1, se muestra el mapa de la memoria RAM de este programa.

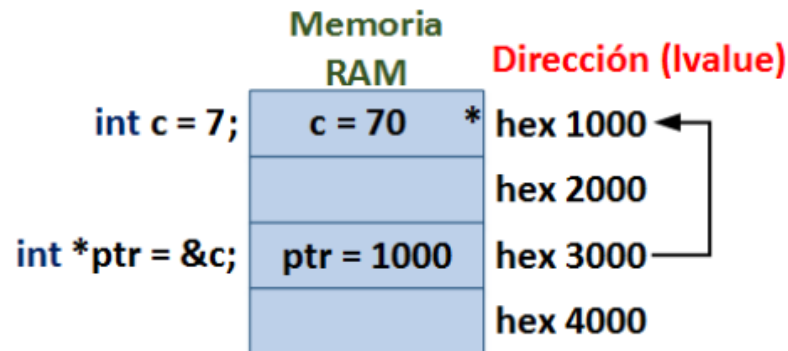


Figura 5.12.1 Mapa de la memoria RAM. Fuente: Zambrano M., et al., 2025.

Problema 5.12

En la Figura 5.12.2, se muestra el diagrama de bloques de los valores 'rvalues' y 'lvalues' de este programa, donde se puede ver claramente que el puntero 'ptr' modifica el contenido de la variable 'c'.

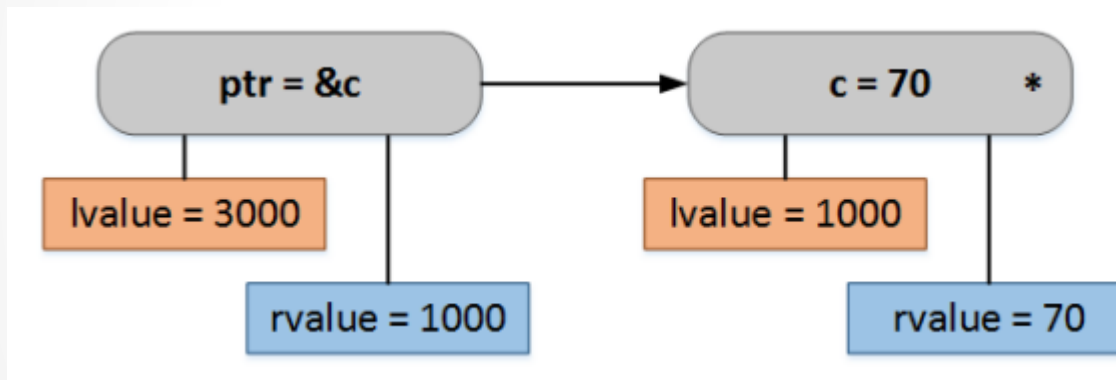


Figura 5.12.1 Mapa de la memoria RAM. Fuente: Zambrano M., et al., 2025.

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- Aplicaciones de Funciones Matemáticas

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- **Paso de Parámetros por Dirección o Punteros**
 - Función LeerDatos()
 - Función ImprimirDatos()
 - Función InvertirDatos()
 - Aplicaciones de Funciones Matemáticas

Paso de Parámetros por Dirección o Punteros

Se pueden retornar múltiples valores con punteros, por lo que los punteros permiten manipular las variables a las cuales apuntan, lo que se conoce como paso de parámetros por dirección o por punteros (Davies, P., 1995). En esta sección se realizarán algunos ejercicios para realizar operaciones de lectura y cálculos matemáticos para modificar los valores de las variables originales utilizando punteros.

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- **Función LeerDatos()**
- **Función ImprimirDatos()**
- **Función InvertirDatos()**
- Aplicaciones de Funciones Matemáticas

Función LeerDatos()

La Figura 5.13.1 muestra cómo trabaja la función 'LeerDatos()', luego de ser declarada y definida. La llamada a la función 'LeerDatos()' envía dos argumentos por dirección a los parámetros (punteros) de la función, la misma que cumple con las tareas de lectura de datos y no retorna ningún valor.

Función LeerDatos()

La llamada a la función hace que se pasen los argumentos que son la dirección de la variable 'a' y la dirección de la variable 'b' dentro de los parámetros de la función que son 2 punteros que reciben las direcciones de las variables 'a' y 'b' respectivamente. La función utiliza estas entradas para ejecutar las tareas de lectura, donde cada puntero accede al contenido de cada variable y modifica su valor.

```
void main()
{
    int a;
    int b;

    LeerDatos(&a, &b);
    cout << endl;
    cout << "Impresión de los números "
         << "originales: " << endl;
    ImprimirDatos(a, b);
    cout << endl;
    Invertir(&a, &b);
    cout << "Impresión de los números "
         << "invertidos: " << endl;
    ImprimirDatos(a, b);

    system("pause");
}
```

```
void LeerDatos(int* ptr_x, int* ptr_y)
{
    cout << "Ingrese el valor del "
         << "primer número: ";
    cin >> *ptr_x;
    cout << "Ingrese el valor del "
         << "segundo número: ";
    cin >> *ptr_y;
}
```

Función que no retorna ningún valor.

Cuerpo de la función que ejecuta la tarea de lectura de dos datos, utilizando punteros.

Figura 5.12.1 . Llamada a la función con parámetros y retorno de un void. Fuente: Zambrano M., et al., 2025.

Función LeerDatos()

En la Figura 5.13.1 se presenta el mapa de la memoria RAM de la función 'LeerDatos()', antes de que los punteros modifiquen los valores contenidos en las variables 'a' y 'b'.

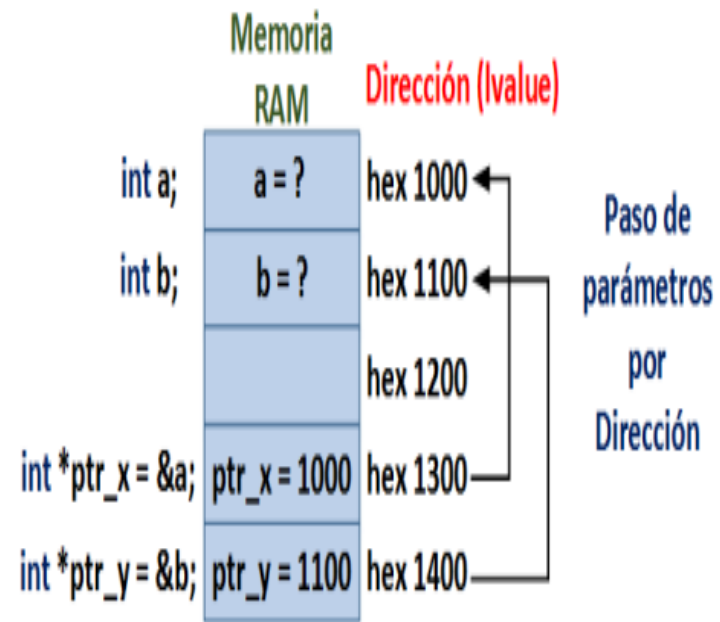


Figura 5.12.1 . Mapa de la memoria RAM antes de modificar los valores de las variables.Fuente: Zambrano M., et al., 2025.

Función LeerDatos()

En la Figura 5.13.2 se presenta el mapa de la memoria RAM de la función 'LeerDatos()', después de que los punteros modifican los valores contenidos en las variables 'a' y 'b', donde cada puntero accede al contenido de cada variable utilizando el operador de indirección (*)

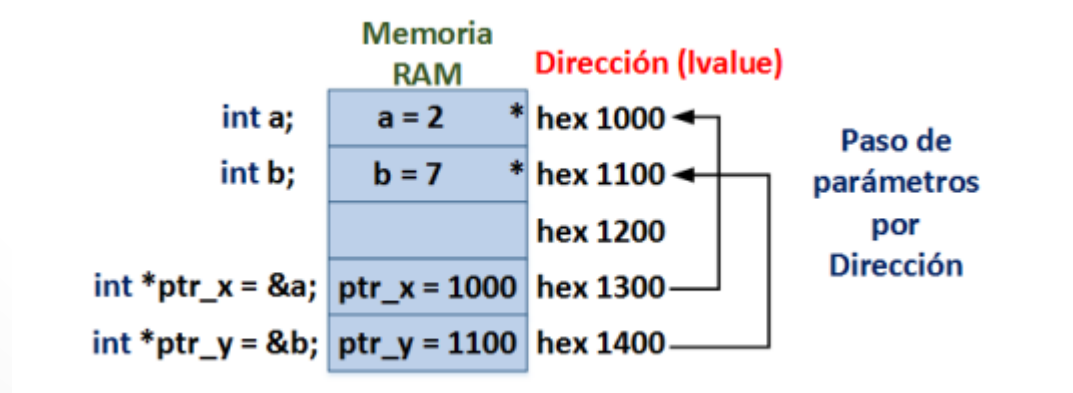
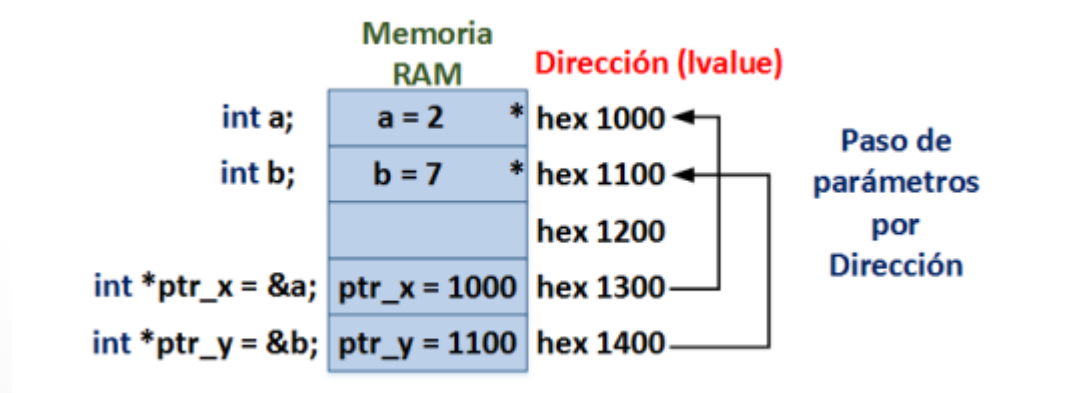


Figura 5.12.12 . Mapa de la memoria RAM después de modificar los valores de las variables. Fuente: Zambrano M., et al., 2025.

Función LeerDatos()

En la Figura 5.13.2 se presenta el mapa de la memoria RAM de la función 'LeerDatos()', después de que los punteros modifican los valores contenidos en las variables 'a' y 'b', donde cada puntero accede al contenido de cada variable utilizando el operador de indirección (*)



Función LeerDatos()

La llamada a la función hace que se pasen dos argumentos que son la variable 'a' y la variable 'b' dentro de los parámetros de la función que son 2 variables 'x' e 'y', que reciben por copia los valores de los argumentos respectivamente. La función utiliza estas entradas para ejecutar las tareas de impresión de datos, donde cada parámetro imprime un valor numérico.

```
void main()
{
    int a;
    int b;

    LeerDatos(&a, &b);
    cout << endl;
    cout << "Impresión de los números "
         << "originales: " << endl;
    ImprimirDatos(a, b);
    cout << endl;
    Invertir(&a, &b);
    cout << "Impresión de los números "
         << "invertidos: " << endl;
    ImprimirDatos(a, b);

    system("pause");
}
```

```
void ImprimirDatos(int x, int y)
{
    cout << "Primer número: "
         << x << endl;
    cout << "Segundo número: "
         << y << endl;
}
```

Función que no retorna ningún valor.

Cuerpo de la función que ejecuta la tarea de impresión de datos, utilizando paso de parámetros por valor.

Figura 5.13.3. Llamada a la función con parámetros y retorno de un void.

Función

ImprimirDatos()

La Figura 5.13.3 muestra cómo trabaja la función 'ImprimirDatos()', luego de ser declarada y definida. La llamada a la función 'ImprimirDatos()' envía dos argumentos por valor a los parámetros de la función, la misma que cumple con las tareas de impresión de datos y no retorna ningún valor. En este caso la función 'ImprimirDatos()', imprime los valores de los números originales antes de ser invertidos.

En la Figura 5.13.4 se presenta el mapa de la memoria RAM de la función 'ImprimirDatos()', donde las variables 'x' e 'y' reciben los valores por copia de las variables 'a' y 'b'.

	Memoria RAM	Dirección (lvalue)	
int a;	a = 2	hex 1000	
int b;	b = 7	hex 1100	
	...	...	
int x = a;	x = 2	hex 1600	Paso de parámetros por Valor
int y = b;	y = 7	hex 1700	

Figura 5.13.4. Mapa de la memoria RAM de la función 'ImprimirDatos()'.

Función ImprimirDatos()

La Figura 5.13.5 muestra el proceso de impresión de los valores de los números invertidos, para lo cual se llama a la función 'ImprimirDatos()' la misma que envía dos argumentos por valor a los parámetros de la función.

```
void main()
{
    int a;
    int b;
```

```
    LeerDatos(&a, &b);
    cout << endl;
    cout << "Impresión de los números "
         << "originales: " << endl;
    ImprimirDatos(a, b);
    cout << endl;
    Invertir(&a, &b);
    cout << "Impresión de los números "
         << "invertidos: " << endl;
    ImprimirDatos(a, b);

    system("pause");
}
```

La llamada a la función hace que se pasen dos argumentos que son la variable 'a' y la variable 'b' dentro de los parámetros de la función que son 2 variables 'x' e 'y', que reciben por copia los valores de los argumentos respectivamente. La función utiliza estas entradas para ejecutar las tareas de impresión de datos, donde cada parámetro imprime un valor numérico.

```
void ImprimirDatos(int x, int y)
{
    cout << "Primer número: "
         << x << endl;
    cout << "Segundo número: "
         << y << endl;
}
```

Función que no retorna ningún valor.

Cuerpo de la función que ejecuta la tarea de impresión de datos, utilizando paso de parámetros por valor.

Figura 5.13.5. Llamada a la función con parámetros y retorno de un void.

Función ImprimirDatos()

En la Figura 5.13.6 se presenta el mapa de la memoria RAM de la función 'ImprimirDatos()', donde las variables 'x' e 'y' reciben los valores por copia de las variables 'a' y 'b'.

	Memoria RAM	Dirección (lvalue)	
int a;	a = 7	hex 1000	
int b;	b = 2	hex 1100	
	...	...	
int x = a;	x = 7	hex 2300	Paso de parámetros por Valor
int y = b;	y = 2	hex 2400	

Figura 5.13.6. Mapa de la memoria RAM de la función 'ImprimirDatos()'.

Función InvertirDatos()

La Figura 5.13.7 muestra cómo trabaja la función 'InvertirDatos()', luego de ser declarada y definida. La llamada a la función 'InvertirDatos()' envía dos argumentos por dirección a los parámetros (punteros) de la función, la misma que cumple con la tarea de invertir los datos y no retorna ningún valor.

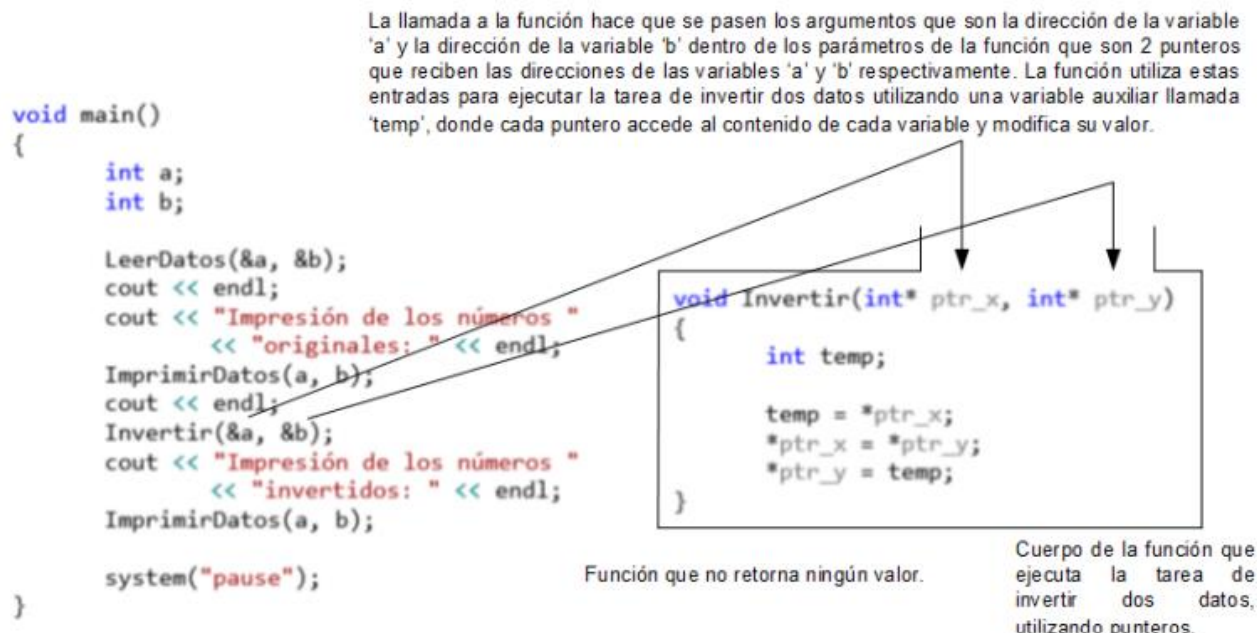


Figura 5.13.7. Llamada a la función con parámetros y retorno de un void.

Función InvertirDatos()

En la Figura 5.13.8 se presenta el mapa de la memoria RAM de la función 'InvertirDatos()', antes de que los punteros modifiquen los valores contenidos en las variables 'a' y 'b', con la ayuda de la variable 'temp'.

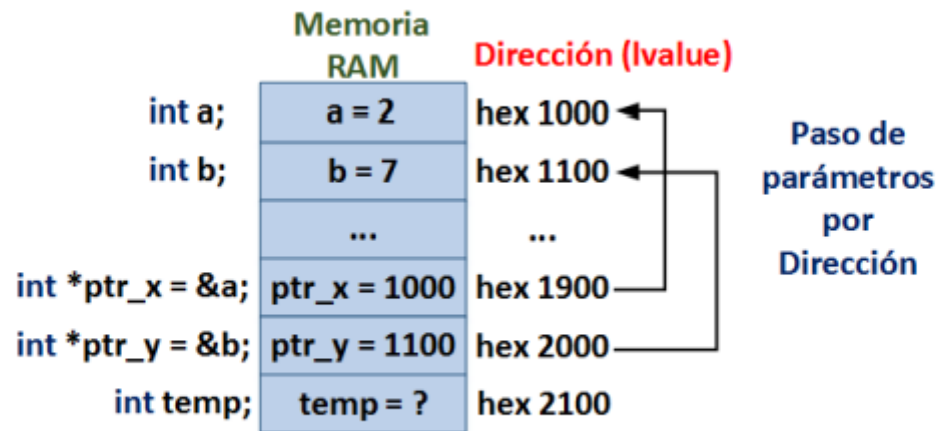


Figura 5.13.8. Mapa de la memoria RAM antes de modificar los valores de las variables.

Función InvertirDatos()

En la Figura 5.13.9 se presenta el mapa de la memoria RAM de la función 'InvertirDatos()', después de que los punteros modifican los valores contenidos en las variables 'a' y 'b', con la ayuda de la variable 'temp', donde cada puntero accede al contenido de cada variable utilizando el operador de indirección (*).

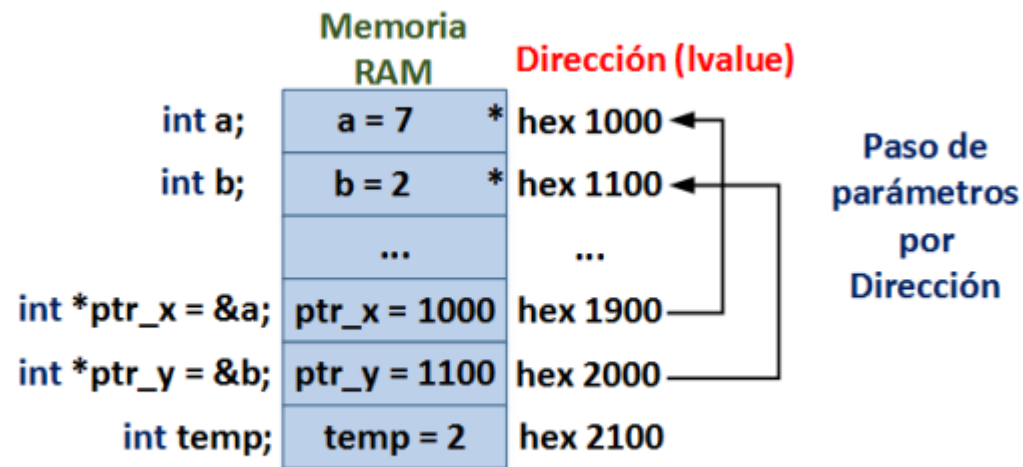


Figura 5.13.9. Mapa de la memoria RAM después de modificar los valores de las variables.

Función InvertirDatos()

En la Figura 5.13.9 se presenta el mapa de la memoria RAM de la función 'InvertirDatos()', después de que los punteros modifican los valores contenidos en las variables 'a' y 'b', con la ayuda de la variable 'temp', donde cada puntero accede al contenido de cada variable utilizando el operador de indirección (*).

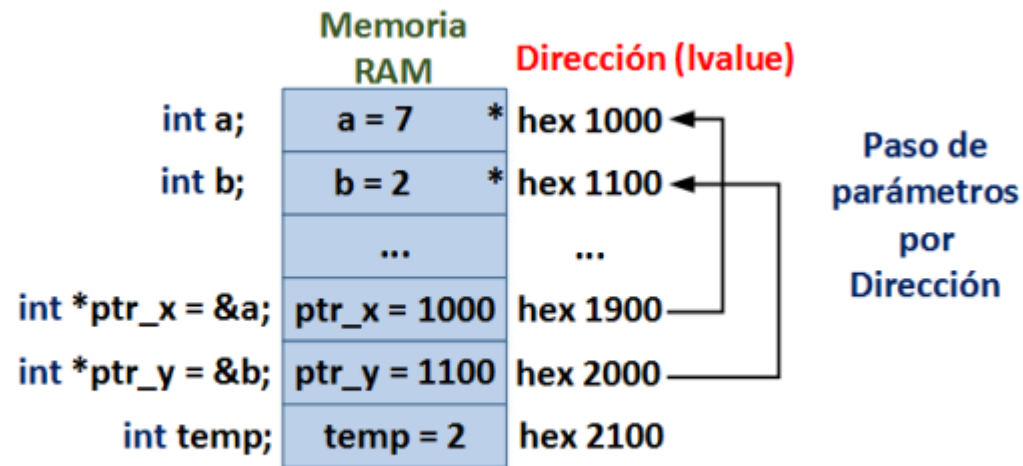


Figura 5.13.9. Mapa de la memoria RAM después de modificar los valores de las variables.

Contenido

- Punteros
- El Operador de Dirección (&)
- El Operador de Indirección (*)
- El 'rvalue' y el 'lvalue' de una Variable y de un Puntero
- Problema 5.12
- Paso de Parámetros por Dirección o Punteros
- Función LeerDatos()
- Función ImprimirDatos()
- Función InvertirDatos()
- **Aplicaciones de Funciones Matemáticas**

Aplicaciones de Funciones Matemáticas

Así, como existe una gran variedad de funciones matemáticas en la librería de Encabezado de Funciones Matemáticas (Math Library Header), como se mencionó en la sección 1.2.1 y luego se aplicó varias de estas funciones en la resolución de problemas como se puede ver en la sección 1.8.1, este número de funciones son limitadas, por lo que se pueden crear librerías propias de funciones matemáticas aplicadas a la vida real y a otras áreas de las ciencias del saber.

Uno de los conceptos más importantes en las Matemáticas es la función, ya que se puede aplicar en numerosas situaciones de la vida cotidiana y determinar las relaciones que existen entre magnitudes tanto en Matemáticas, Física, Química, Biología, Economía, etc., y poder calcular el valor de una de ellas en función de otras de las que depende (Ballester, S., 2009).

El famoso matemático Euler fue el primero en emplear la expresión $y = f(x)$ para representar una función 'f' asociada a un valor 'x'. Con esta representación que es empleada hoy en el Cálculo Diferencial e Integral, se comenzó a utilizar el concepto de función en las Matemáticas. En esta sección se presentarán algunas funciones aplicadas a varias áreas del saber tales como: Cinemática, Dinámica, Energía, Química, Acústica, Economía, Termodinámica y Física Clásica.

Pensamiento

Científico de la Computación

- “Todo el mundo debería saber programar”

Steve Jobs
1955-2011

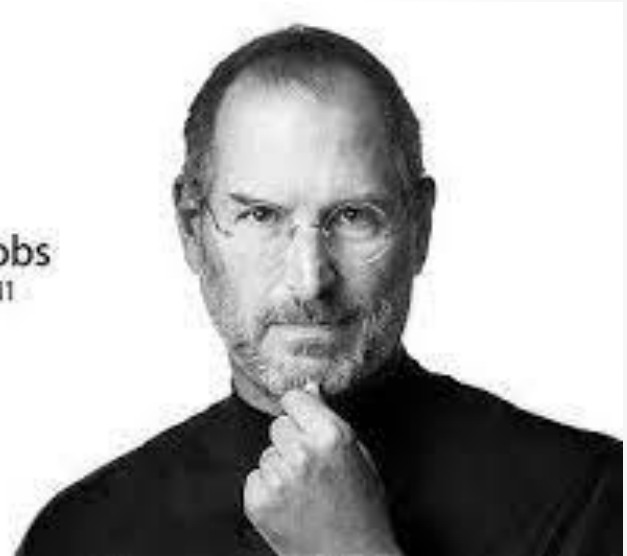


Figura 32. Universidad de las Fuerzas Armadas - ESPE.
Fuente: <https://economipedia.com/definiciones/steve-jobs.html>

¿Preguntas?

!Gracias!